



COMP3153/9153

Algorithmic Verification

Lecture 4: Symbolic CTL Model Checking

Symbolic Model Checking

Model checking so far...

To check if $\mathcal{K} \models \varphi$:

- Check if φ holds in s_0

Symbolic Model Checking

Model checking so far...

To check if $\mathcal{K} \models \varphi$:

- Check if φ holds in s_0
- e.g. If $\varphi = \mathbf{E X} \psi$:
 - Choose a successor s_1 of s_0 ,
 - Check if ψ holds in s_1 ,
 - etc

Symbolic Model Checking

Model checking so far...

To check if $\mathcal{K} \models \varphi$:

- Check if φ holds in s_0
- e.g. If $\varphi = \mathbf{E X} \psi$:
 - Choose a successor s_1 of s_0 ,
 - Check if ψ holds in s_1 ,
 - etc

Symbolic Model Checking

Rather than evaluating formulae at individual states, look at the picture as a whole.

“Evaluate” formulae **symbolically** (abstractly) first, then bring down to a concrete level.

Symbolic Model Checking

Model checking so far...

To check if $\mathcal{K} \models \varphi$:

- Check if φ holds in s_0
- e.g. If $\varphi = \mathbf{E X} \psi$:
 - Choose a successor s_1 of s_0 ,
 - Check if ψ holds in s_1 ,
 - etc

Symbolic Model Checking

Rather than evaluating formulae at individual states, look at the picture as a whole.

“Evaluate” formulae **symbolically** (abstractly) first, then bring down to a concrete level.

Can simplify model checking by identifying optimization areas such as duplication or unnecessary computations.

Symbolic Model Checking

Key idea

Associate formulae with **sets of states**.

Use lattice theory to evaluate symbolically.

Math Recap: Partial orders and Lattices

Definition

A *partial order*, $\preceq$, is a binary relation which is:

- Reflexive $(x \preceq x \text{ for all } x)$
- Anti-symmetric $(x \preceq y \wedge y \preceq x \Rightarrow x = y)$
- Transitive $(x \preceq y \wedge y \preceq z \Rightarrow x \preceq z)$

Definition

A *lattice* $(L, \preceq)$ is a set L and a partial order $(\preceq) \subseteq L \times L$ where any set $X \subseteq L$ has both a *least upper bound* $\sup X \in L$ and a *greatest lower bound* $\inf X \in L$.

We define $\top = \sup L$ and $\perp = \inf L$

Fixed points

If we have a function $f : L \rightarrow L$ that is *monotone*, i.e:

$$x \preceq y \Rightarrow f(x) \preceq f(y)$$

Fixed points

If we have a function $f : L \rightarrow L$ that is *monotone*, i.e:

$$x \preceq y \Rightarrow f(x) \preceq f(y)$$

I can prove that f has both a *least* and *greatest fixed point*, i.e.

Definition

Least The *least fixed point* $\mu.f$ of a function $f : L \rightarrow L$ is the smallest (wrt. $\preceq$) element $x \in L$ such that $f(x) = x$.

Greatest The *greatest fixed point* $\nu.f$ of a function $f : L \rightarrow L$ is the largest (wrt. $\preceq$) element $x \in L$ s.t. $f(x) = x$.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$,

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$,

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$,

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.
- Thus $x \preceq f(x) \preceq f(u)$. So $f(u)$ is also an upper bound of D .

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.
- Thus $x \preceq f(x) \preceq f(u)$. So $f(u)$ is also an upper bound of D .
- u is the least upper bound of D , so $u \preceq f(u)$.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.
- Thus $x \preceq f(x) \preceq f(u)$. So $f(u)$ is also an upper bound of D .
- u is the least upper bound of D , so $u \preceq f(u)$. Thus $u \in D$.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.
- Thus $x \preceq f(x) \preceq f(u)$. So $f(u)$ is also an upper bound of D .
- u is the least upper bound of D , so $u \preceq f(u)$. Thus $u \in D$.
- By monotonicity, $f(u) \preceq f(f(u))$,

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.
- Thus $x \preceq f(x) \preceq f(u)$. So $f(u)$ is also an upper bound of D .
- u is the least upper bound of D , so $u \preceq f(u)$. Thus $u \in D$.
- By monotonicity, $f(u) \preceq f(f(u))$, so $f(u) \in D$

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.
- Thus $x \preceq f(x) \preceq f(u)$. So $f(u)$ is also an upper bound of D .
- u is the least upper bound of D , so $u \preceq f(u)$. Thus $u \in D$.
- By monotonicity, $f(u) \preceq f(f(u))$, so $f(u) \in D$
- Because u is the least upper bound of D , $f(u) \preceq u$.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.
- Thus $x \preceq f(x) \preceq f(u)$. So $f(u)$ is also an upper bound of D .
- u is the least upper bound of D , so $u \preceq f(u)$. Thus $u \in D$.
- By monotonicity, $f(u) \preceq f(f(u))$, so $f(u) \in D$
- Because u is the least upper bound of D , $f(u) \preceq u$.
- Therefore $f(u) = u$, i.e. u is a fixed point.

Knaster-Tarski Theorem

Let's prove it for greatest fixed points, given a lattice L and a monotone function f :

- Define $D = \{x \in L \mid x \preceq f(x)\}$.
- We know that $\forall m. \perp \preceq m$, so we know $\perp \in D$, and, by monotonicity, $f(\perp) \in D$, $f(f(\perp)) \in D$ etc.
- Let $u = \sup D$, the least upper bound.
- Hence for all $x \in D$, $x \preceq u$, and by monotonicity $f(x) \preceq f(u)$.
- Thus $x \preceq f(x) \preceq f(u)$. So $f(u)$ is also an upper bound of D .
- u is the least upper bound of D , so $u \preceq f(u)$. Thus $u \in D$.
- By monotonicity, $f(u) \preceq f(f(u))$, so $f(u) \in D$
- Because u is the least upper bound of D , $f(u) \preceq u$.
- Therefore $f(u) = u$, i.e. u is a fixed point.
- All fixed points are in D , therefore u is the greatest fixed point.

Finding Fixed Points

How do we compute fixed points?

Finding Fixed Points

How do we compute fixed points?

For *finite lattices*, we can compute the least fixed point by *iterating* f from $\perp$, and the greatest by iterating from $\top$:

Specifically, define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

Claim

$\lim_{i \rightarrow \infty} x_i$ exists, and is equal to $\nu.f$

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.
- By induction, $x_i \succeq x_{i+1}$ for all i .

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.
- By induction, $x_i \succeq x_{i+1}$ for all i .
- Because L is finite, there exists j, k such that $j < k$ and $x_j = x_k$.

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.
- By induction, $x_i \succeq x_{i+1}$ for all i .
- Because L is finite, there exists j, k such that $j < k$ and $x_j = x_k$.
- By transitivity of $\preceq$, we have $x_j = x_{j+1} = \dots = x_k$; and by induction $x_i = x_j$ for all $i > j$.

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.
- By induction, $x_i \succeq x_{i+1}$ for all i .
- Because L is finite, there exists j, k such that $j < k$ and $x_j = x_k$.
- By transitivity of $\succeq$, we have $x_j = x_{j+1} = \dots = x_k$; and by induction $x_i = x_j$ for all $i > j$.
- So $\lim_{i \rightarrow \infty} x_i$ exists and is equal to x_j .

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.
- By induction, $x_i \succeq x_{i+1}$ for all i .
- Because L is finite, there exists j, k such that $j < k$ and $x_j = x_k$.
- By transitivity of $\preceq$, we have $x_j = x_{j+1} = \dots = x_k$; and by induction $x_i = x_j$ for all $i > j$.
- So $\lim_{i \rightarrow \infty} x_i$ exists and is equal to x_j .
- $x_0 = \top$ is an upper bound for $D = \{x \in L \mid x \preceq f(x)\}$, so $\nu.f \preceq x_0$.

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.
- By induction, $x_i \succeq x_{i+1}$ for all i .
- Because L is finite, there exists j, k such that $j < k$ and $x_j = x_k$.
- By transitivity of $\preceq$, we have $x_j = x_{j+1} = \dots = x_k$; and by induction $x_i = x_j$ for all $i > j$.
- So $\lim_{i \rightarrow \infty} x_i$ exists and is equal to x_j .
- $x_0 = \top$ is an upper bound for $D = \{x \in L \mid x \preceq f(x)\}$, so $\nu.f \preceq x_0$.
- By monotonicity, $\nu.f = f(\nu.f) \preceq f(x_0) = x_1$.

Knaster-Tarski Theorem II

Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.
- By induction, $x_i \succeq x_{i+1}$ for all i .
- Because L is finite, there exists j, k such that $j < k$ and $x_j = x_k$.
- By transitivity of $\preceq$, we have $x_j = x_{j+1} = \dots = x_k$; and by induction $x_i = x_j$ for all $i > j$.
- So $\lim_{i \rightarrow \infty} x_i$ exists and is equal to x_j .
- $x_0 = \top$ is an upper bound for $D = \{x \in L \mid x \preceq f(x)\}$, so $\nu.f \preceq x_0$.
- By monotonicity, $\nu.f = f(\nu.f) \preceq f(x_0) = x_1$.
- By induction, $\nu.f \preceq x_i$ for all i . In particular $\nu.f \preceq x_j = \lim_{i \rightarrow \infty} x_i$.

Knaster-Tarski Theorem II

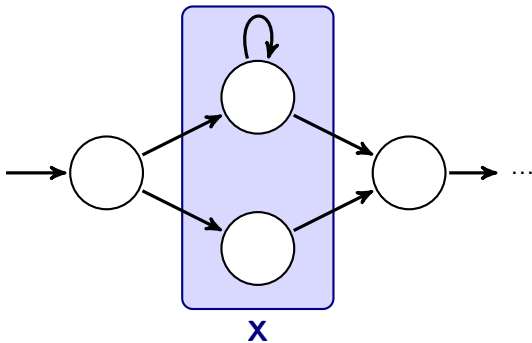
Given a finite lattice L and a monotone function f , define the sequence $x_0, x_1, \dots$ as:

$$x_0 = \top \quad x_{i+1} = f(x_i)$$

- $x_0 = \top \succeq x_1$, so by monotonicity $x_1 = f(x_0) \succeq f(x_1) = x_2$.
- By induction, $x_i \succeq x_{i+1}$ for all i .
- Because L is finite, there exists j, k such that $j < k$ and $x_j = x_k$.
- By transitivity of $\preceq$, we have $x_j = x_{j+1} = \dots = x_k$; and by induction $x_i = x_j$ for all $i > j$.
- So $\lim_{i \rightarrow \infty} x_i$ exists and is equal to x_j .
- $x_0 = \top$ is an upper bound for $D = \{x \in L \mid x \preceq f(x)\}$, so $\nu.f \preceq x_0$.
- By monotonicity, $\nu.f = f(\nu.f) \preceq f(x_0) = x_1$.
- By induction, $\nu.f \preceq x_i$ for all i . In particular $\nu.f \preceq x_j = \lim_{i \rightarrow \infty} x_i$.
- But $f(x_j) = x_{j+1} = x_j$, so $x_j \in D$, and hence $x_j \preceq \nu.f$

Successors and predecessors

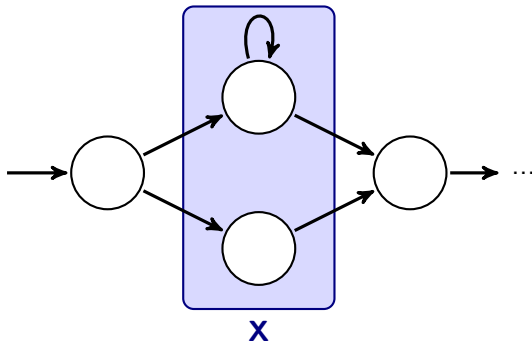
Given an automaton (Q, Σ, δ, q_0) , and $X \subseteq Q$



Successors and predecessors

Given an automaton (Q, Σ, δ, q_0) , and $X \subseteq Q$

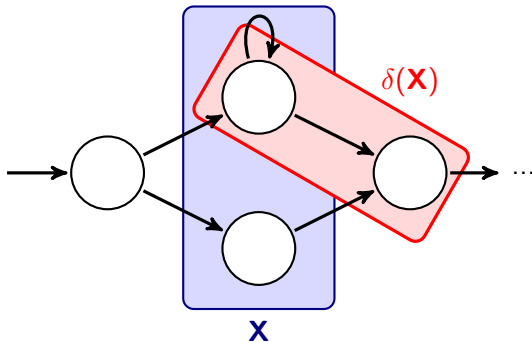
- Successors of X : $\delta(X) = \bigcup_{x \in X} \{x' : x' \in \delta(x)\}$



Successors and predecessors

Given an automaton (Q, Σ, δ, q_0) , and $X \subseteq Q$

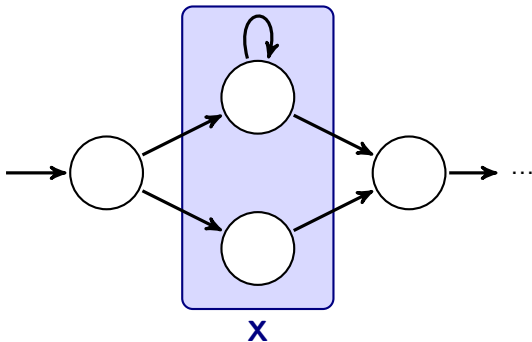
- Successors of X : $\delta(X) = \bigcup_{x \in X} \{x' : x' \in \delta(x)\}$



Successors and predecessors

Given an automaton (Q, Σ, δ, q_0) , and $X \subseteq Q$

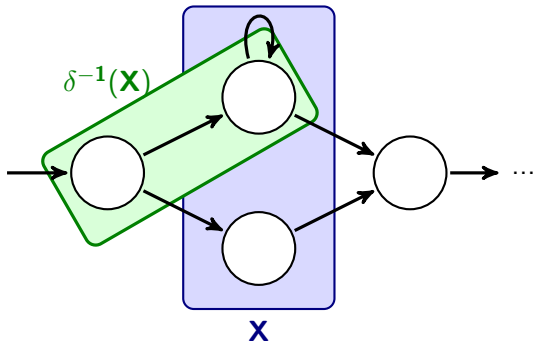
- Successors of X : $\delta(X) = \bigcup_{x \in X} \{x' : x' \in \delta(x)\}$
- Predecessors of X : $\delta^{-1}(X) = \bigcup_{x \in X} \{x' : x \in \delta(x')\}$



Successors and predecessors

Given an automaton (Q, Σ, δ, q_0) , and $X \subseteq Q$

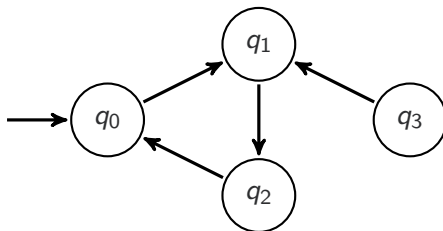
- Successors of X : $\delta(X) = \bigcup_{x \in X} \{x' : x' \in \delta(x)\}$
- Predecessors of X : $\delta^{-1}(X) = \bigcup_{x \in X} \{x' : x \in \delta(x')\}$



Monotone functions: Example 1

Consider the function:

$$f(S) = (\{q_0\} \cup S) \cup \delta(S)$$

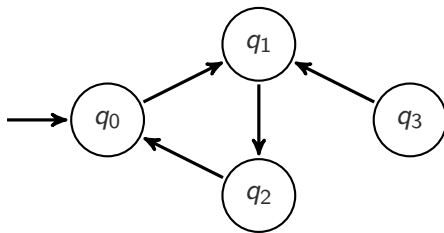


Monotone functions: Example 1

Consider the function:

$$f(S) = (\{q_0\} \cup S) \cup \delta(S)$$

- Is it monotone?

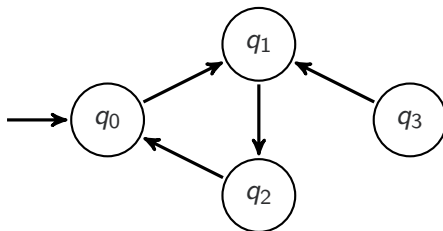


Monotone functions: Example 1

Consider the function:

$$f(S) = (\{q_0\} \cup S) \cup \delta(S)$$

- Is it monotone? ✓

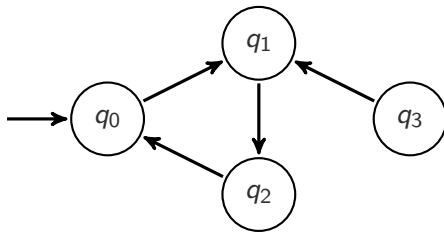


Monotone functions: Example 1

Consider the function:

$$f(S) = (\{q_0\} \cup S) \cup \delta(S)$$

- Is it monotone? ✓
- What is $\mu.f$?
- What is $\nu.f$?



Monotone functions: Example 1

Consider the function:

$$f(S) = (\{q_0\} \cup S) \cup \delta(S)$$

Claim

$$\text{Reach}(\{q_0\}) = \mu.f$$

Proof

Monotone functions: Example 1

Consider the function:

$$f(S) = (\{q_0\} \cup S) \cup \delta(S)$$

Claim

$$\text{Reach}(\{q_0\}) = \mu.f$$

Proof

- $\text{Reach}(\{q_0\})$ is a fixed point

Monotone functions: Example 1

Consider the function:

$$f(S) = (\{q_0\} \cup S) \cup \delta(S)$$

Claim

$$\text{Reach}(\{q_0\}) = \mu.f$$

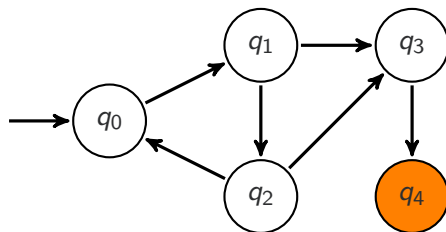
Proof

- $\text{Reach}(\{q_0\})$ is a fixed point
- $\text{Reach}(\{q_0\})$ is a subset of all fixed points

Monotone functions: Example 2

Consider the function:

$$f(S) = (\neg \bullet) \cap \delta^{-1}(S)$$

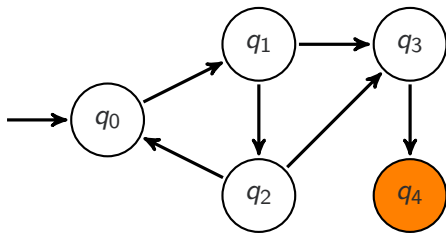


Monotone functions: Example 2

Consider the function:

$$f(S) = (\neg \bullet) \cap \delta^{-1}(S)$$

- Is it monotone?

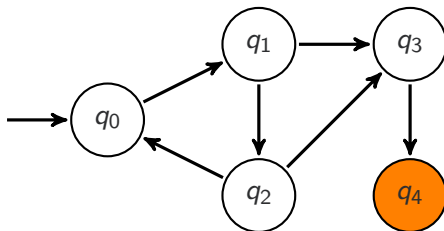


Monotone functions: Example 2

Consider the function:

$$f(S) = (\neg \bullet) \cap \delta^{-1}(S)$$

- Is it monotone? ✓

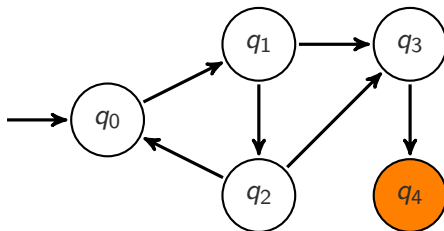


Monotone functions: Example 2

Consider the function:

$$f(S) = (\neg \bullet) \cap \delta^{-1}(S)$$

- Is it monotone? ✓
- What is $\mu.f$?
- What is $\nu.f$?



Monotone functions: Example 2

Consider the function:

$$f(S) = (\neg \bullet) \cap \delta^{-1}(S)$$

Claim

$$\nu.f = \{q : q \models \mathbf{E\,G\,}\neg \bullet\}$$

Proof

Monotone functions: Example 2

Consider the function:

$$f(S) = (\neg \bullet) \cap \delta^{-1}(S)$$

Claim

$$\nu.f = \{q : q \models \mathbf{E\,G\,}\neg \bullet\}$$

Proof

- $\{q : q \models \mathbf{E\,G\,}\neg \bullet\}$ is a fixed point

Monotone functions: Example 2

Consider the function:

$$f(S) = (\neg \bullet) \cap \delta^{-1}(S)$$

Claim

$$\nu.f = \{q : q \models \mathbf{E\,G\,}\neg \bullet\}$$

Proof

- $\{q : q \models \mathbf{E\,G\,}\neg \bullet\}$ is a fixed point
- $\{q : q \models \mathbf{E\,G\,}\neg \bullet\}$ is a superset of all fixed points

CTL Syntax

Recall a CTL formula:

$$\begin{array}{lcl} \varphi := & p & \text{for } p \in \text{PROP} \\ & | & \varphi \wedge \varphi \\ & | & \neg \varphi \\ & | & \mathbf{EX} \varphi \\ & | & \mathbf{AX} \varphi \\ & | & \mathbf{E}(\varphi \mathbf{U} \varphi) \\ & | & \mathbf{A}(\varphi \mathbf{U} \varphi) \end{array}$$

Let $\mathcal{F}_{\text{CTL}}$ denote the set of all CTL formulae.

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket =$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket =$$

$$\llbracket \neg \varphi \rrbracket =$$

$$\llbracket \mathbf{EX} \varphi \rrbracket =$$

$$\llbracket \mathbf{AX} \varphi \rrbracket =$$

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket =$$

$$\llbracket \neg \varphi \rrbracket =$$

$$\llbracket \mathbf{EX} \varphi \rrbracket =$$

$$\llbracket \mathbf{AX} \varphi \rrbracket =$$

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket =$$

$$\llbracket \mathbf{EX} \varphi \rrbracket =$$

$$\llbracket \mathbf{AX} \varphi \rrbracket =$$

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket = Q \setminus \llbracket \varphi \rrbracket$$

$$\llbracket \text{EX} \varphi \rrbracket =$$

$$\llbracket \text{AX} \varphi \rrbracket =$$

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket = Q \setminus \llbracket \varphi \rrbracket = \overline{\llbracket \varphi \rrbracket}$$

$$\llbracket \text{EX} \varphi \rrbracket =$$

$$\llbracket \text{AX} \varphi \rrbracket =$$

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket = Q \setminus \llbracket \varphi \rrbracket = \overline{\llbracket \varphi \rrbracket}$$

$$\llbracket \text{EX} \varphi \rrbracket = \delta^{-1}(\llbracket \varphi \rrbracket)$$

$$\llbracket \text{AX} \varphi \rrbracket =$$

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket = Q \setminus \llbracket \varphi \rrbracket = \overline{\llbracket \varphi \rrbracket}$$

$$\llbracket \mathbf{EX} \varphi \rrbracket = \delta^{-1}(\llbracket \varphi \rrbracket)$$

$$\llbracket \mathbf{AX} \varphi \rrbracket = Q \setminus (\delta^{-1}(Q \setminus \llbracket \varphi \rrbracket))$$

CTL Symbolic Semantics

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket = Q \setminus \llbracket \varphi \rrbracket = \overline{\llbracket \varphi \rrbracket}$$

$$\llbracket \mathbf{EX} \varphi \rrbracket = \delta^{-1}(\llbracket \varphi \rrbracket)$$

$$\llbracket \mathbf{AX} \varphi \rrbracket = Q \setminus (\delta^{-1}(Q \setminus \llbracket \varphi \rrbracket)) = \overline{\delta^{-1}(\overline{\llbracket \varphi \rrbracket})}$$

Dealing with Until

Question

What is an appropriate choice for $\llbracket \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2) \rrbracket$ and $\llbracket \mathbf{A}(\varphi_1 \mathbf{U} \varphi_2) \rrbracket$?

Dealing with Until

Question

What is an appropriate choice for $\llbracket \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2) \rrbracket$ and $\llbracket \mathbf{A}(\varphi_1 \mathbf{U} \varphi_2) \rrbracket$?

Observation

$$\mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2) \iff \begin{cases} \mathcal{K}, q \models \varphi_2, \text{ or} \\ \mathcal{K}, q \models \varphi_1 \text{ and } \mathcal{K}, q \models \mathbf{EXE}(\varphi_1 \mathbf{U} \varphi_2) \end{cases}$$

Dealing with Until

Question

What is an appropriate choice for $\llbracket \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2) \rrbracket$ and $\llbracket \mathbf{A}(\varphi_1 \mathbf{U} \varphi_2) \rrbracket$?

Observation

$$\mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2) \iff \begin{cases} \mathcal{K}, q \models \varphi_2, \text{ or} \\ \mathcal{K}, q \models \varphi_1 \text{ and } \mathcal{K}, q \models \mathbf{EXE}(\varphi_1 \mathbf{U} \varphi_2) \end{cases}$$

Consider:

$$f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)}(S) = \llbracket \varphi_2 \rrbracket \cup (\llbracket \varphi_1 \rrbracket \cap \delta^{-1}(S))$$

Dealing with Until

Claim

$\mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)$ if and only if $q \in \mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)}$

Dealing with Until

Claim

$\mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)$ if and only if $q \in \mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)}$

Proof

Let $S = \{q : \mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)\}$.

Dealing with Until

Claim

$\mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)$ if and only if $q \in \mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)}$

Proof

Let $S = \{q : \mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)\}$.

- $\mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)} \subseteq S$: Show that S is a fixed point:

Dealing with Until

Claim

$\mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)$ if and only if $q \in \mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)}$

Proof

Let $S = \{q : \mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)\}$.

- $\mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)} \subseteq S$: Show that S is a fixed point:
 - $f(S) \subseteq S$

Dealing with Until

Claim

$\mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)$ if and only if $q \in \mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)}$

Proof

Let $S = \{q : \mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)\}$.

- $\mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)} \subseteq S$: Show that S is a fixed point:
 - $f(S) \subseteq S$
 - $S \subseteq f(S)$

Dealing with Until

Claim

$\mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)$ if and only if $q \in \mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)}$

Proof

Let $S = \{q : \mathcal{K}, q \models \mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)\}$.

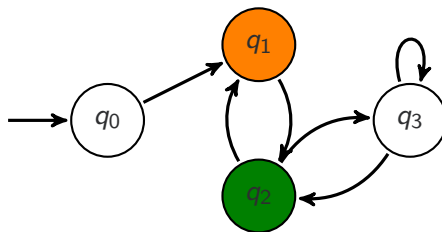
- $\mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)} \subseteq S$: Show that S is a fixed point:
 - $f(S) \subseteq S$
 - $S \subseteq f(S)$
- $S \subseteq \mu.f_{\mathbf{E}(\varphi_1 \mathbf{U} \varphi_2)}$: Prove that $S \subseteq Y$ for any fixed point Y

Example

$$\varphi = \mathbf{E} \bullet \mathbf{U} \bullet$$

$$f_{\varphi}(S) = \llbracket \bullet \rrbracket \cup (\llbracket \bullet \rrbracket \cap \delta^{-1}(S))$$

- Compute $\mu.f_{\varphi}$

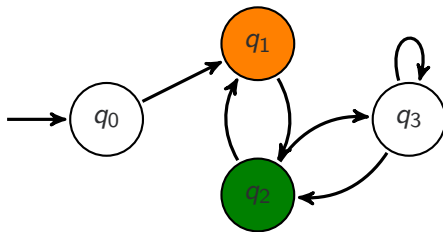


Example

$$\varphi = \mathbf{E} \bullet \mathbf{U} \bullet$$

$$f_{\varphi}(S) = \llbracket \bullet \rrbracket \cup (\llbracket \bullet \rrbracket \cap \delta^{-1}(S))$$

- Compute $\mu.f_{\varphi}$
- What is $\nu.f_{\varphi}$?



CTL Symbolic Semantics (ctd)

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket = Q \setminus \llbracket \varphi \rrbracket = \overline{\llbracket \varphi \rrbracket}$$

$$\llbracket \text{EX} \varphi \rrbracket = \delta^{-1}(\llbracket \varphi \rrbracket)$$

$$\llbracket \text{AX} \varphi \rrbracket = Q \setminus (\delta^{-1}(Q \setminus \llbracket \varphi \rrbracket)) = \overline{\delta^{-1}(\overline{\llbracket \varphi \rrbracket})}$$

$$\llbracket \text{E}(\varphi_1 \text{ U } \varphi_2) \rrbracket =$$

$$\llbracket \text{A}(\varphi_1 \text{ U } \varphi_2) \rrbracket =$$

CTL Symbolic Semantics (ctd)

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket = Q \setminus \llbracket \varphi \rrbracket = \overline{\llbracket \varphi \rrbracket}$$

$$\llbracket \text{EX} \varphi \rrbracket = \delta^{-1}(\llbracket \varphi \rrbracket)$$

$$\llbracket \text{AX} \varphi \rrbracket = Q \setminus (\delta^{-1}(Q \setminus \llbracket \varphi \rrbracket)) = \overline{\delta^{-1}(\overline{\llbracket \varphi \rrbracket})}$$

$$\llbracket \text{E}(\varphi_1 \text{ U } \varphi_2) \rrbracket = \mu X. \llbracket \varphi_2 \rrbracket \cup (\llbracket \varphi_1 \rrbracket \cap \delta^{-1}(X))$$

$$\llbracket \text{A}(\varphi_1 \text{ U } \varphi_2) \rrbracket =$$

CTL Symbolic Semantics (ctd)

Definition

Given a Kripke structure $\mathcal{K} = (Q, \delta, q_0, \mathcal{L})$, define

$$\llbracket \cdot \rrbracket : \mathcal{F}_{\text{CTL}} \rightarrow 2^Q$$

recursively as follows:

$$\llbracket p \rrbracket = \{q : p \in \mathcal{L}(q)\}$$

$$\llbracket \varphi_1 \wedge \varphi_2 \rrbracket = \llbracket \varphi_1 \rrbracket \cap \llbracket \varphi_2 \rrbracket$$

$$\llbracket \neg \varphi \rrbracket = Q \setminus \llbracket \varphi \rrbracket = \overline{\llbracket \varphi \rrbracket}$$

$$\llbracket \text{EX} \varphi \rrbracket = \delta^{-1}(\llbracket \varphi \rrbracket)$$

$$\llbracket \text{AX} \varphi \rrbracket = Q \setminus (\delta^{-1}(Q \setminus \llbracket \varphi \rrbracket)) = \overline{\delta^{-1}(\overline{\llbracket \varphi \rrbracket})}$$

$$\llbracket \text{E}(\varphi_1 \text{ U } \varphi_2) \rrbracket = \mu X. \llbracket \varphi_2 \rrbracket \cup (\llbracket \varphi_1 \rrbracket \cap \delta^{-1}(X))$$

$$\llbracket \text{A}(\varphi_1 \text{ U } \varphi_2) \rrbracket = \mu X. \llbracket \varphi_2 \rrbracket \cup (\llbracket \varphi_1 \rrbracket \cap \overline{\delta^{-1}(\overline{X})})$$

CTL Symbolic Semantics (ctd)

Semantics of derived operators

$$\llbracket \mathbf{EF}\varphi \rrbracket = \mu X. \llbracket \varphi \rrbracket \cup \delta^{-1}(X)$$

$$\llbracket \mathbf{EG}\varphi \rrbracket = \nu X. \llbracket \varphi \rrbracket \cap \delta^{-1}(X)$$

$$\llbracket \mathbf{AF}\varphi \rrbracket = \nu X. \llbracket \varphi \rrbracket \cap \overline{\delta^{-1}(\overline{X})}$$

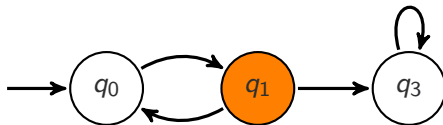
$$\llbracket \mathbf{AG}\varphi \rrbracket = \mu X. \llbracket \varphi \rrbracket \cup \overline{\delta^{-1}(\overline{X})}$$

Example

$$\varphi = \mathbf{EF} \bullet$$

$$f_{\varphi}(S) = \llbracket \bullet \rrbracket \cup \delta^{-1}(S)$$

- Compute $\mu.f_{\varphi} = \llbracket \mathbf{EF} \bullet \rrbracket$

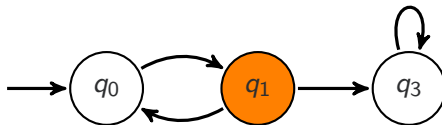


Example

$$\varphi = \mathbf{EF} \bullet$$

$$f_{\varphi}(S) = \llbracket \bullet \rrbracket \cup \delta^{-1}(S)$$

- Compute $\mu.f_{\varphi} = \llbracket \mathbf{EF} \bullet \rrbracket$
- What is $\nu.f_{\varphi}$?



Bibliography

Fixed-point basics:

- Grädel et al: Finite Model Theory, Section 3.3.1

Symbolic Model Checking:

- McMillan: Symbolic Model Checking

Weekly Feedback

I would appreciate any comments/suggestions/requests you have on this week's lectures (and tutorials).



<https://forms.office.com/r/sCi2xjavrS>